AI Usage Log

QA Take-Home Assessment — Whole-Assessment AI Usage Record
Author: Uma Uka | Date: 31 August 2026

Scope of this log: Claude Code (Anthropic) was used as an interactive pair-programming assistant for Part 2 (API test automation) and Part 5 (CI/CD), across a single continuous session. This log records that session's real prompts, actions, and corrections. Part 1 (test plan/test cases) and Part 6 Task A (AI critique) involved AI assistance in a separate tool/session not visible here — that usage should be appended separately by the author rather than reconstructed secondhand, to keep this log limited to what can be honestly verified.

1. What AI Was Used For

- Scaffolding the Playwright API test suite architecture for Part 2 (fixtures/helpers/config/tests split) from a detailed brief, with an explicit planning step and sign-off before any code was written.
- Iteratively building each spec file (auth, CRUD, negative/boundary, filtering) one at a time, verifying real behavior against the running local Restful Booker instance via direct HTTP requests before writing any assertion — so test expectations were based on observed behavior, not assumed "typical REST API" behavior.
- Designing and building the GitHub Actions CI/CD workflow for Part 5: job parallelization, the Restful Booker service container, test sharding, artifact upload, and the `test.fail()` known-bug CI signal — then verifying the actual workflow run on GitHub Actions job-by-job, not just trusting the committed YAML.
- Generating a formatted PDF bug report (an initial docs/Bug-Report.pdf, later superseded by the author's own docs/Restful_Booker_Bug_Report.pdf) via a small Python/ReportLab script.
- Drafting and maintaining README.md, including catching and fixing a stale file-path reference after a later document rename.
- General git workflow support: staging, committing, and pushing in small increments after each reviewed unit of work, per an explicit process specified by the author.

2. Example Prompts

Prompt 1 — Initial scope-setting

> *"I'm building the API test automation suite (Part 2) and CI/CD pipeline (Part 5) for a QA engineer take-home assessment. Read this whole brief, then STOP and show me your planned file structure and test list before writing any code — I need to sign off on the approach first..."*

What I did with it: Produced a full plan (file structure, test list per spec file, dependency choices) and entered a review gate before writing any code, rather than generating the suite in one pass.

Prompt 2 — Correcting the tech stack and workflow cadence

> *“I'm using JavaScript not TypeScript. I want to commit each spec/test grouping to the repo so I will need you to pause and commit each after each. eg. when we do auth part we commit and push to remote repo.”*

What I did with it: Revised the plan from TypeScript to plain JavaScript, and switched to committing and pushing after each spec file was built and verified, rather than one final commit at the end.

Prompt 3 — Requesting a formatted deliverable

> *“Yes, draft the Bug-Report content with severities and put it in a well formatted pdf file”*

What I did with it: Assigned severities to each defect with stated reasoning, then wrote and ran a Python/ReportLab script to produce a formatted PDF (colored severity chips, code blocks for requests, an appendix for the verified-safe finding), and visually inspected the rendered pages before presenting it.

Prompt 4 — Formatting correction

> *“Could you change § to section and give a line space between Bug Report heading and the rest of the document”*

What I did with it: Located the single § occurrence and the title-spacing issue in the PDF generation script, fixed both, regenerated the PDF, and re-verified visually before confirming.

3. What Was Corrected

- **Tech stack assumption:** the initial plan followed the brief's mention of TypeScript; the author corrected this to plain JavaScript before any code was written.
- **Delivery cadence:** the default approach would have been to build the whole suite then commit once; the author required committing and pushing after each reviewed spec file instead.
- **A self-caught testing gap:** local manual test runs used a `--reporter=list` override for cleaner terminal output, which silently skipped the JSON/JUnit reporters defined in the actual config. This was caught before trusting the CI behavior, by re-running the suite without the override and confirming the report files were produced as configured.
- **A stale doc reference:** the README pointed to `docs/Bug-Report.pdf` after the author replaced that file with their own `Restful_Booker_Bug_Report.pdf`. Caught when asked directly whether the README was still accurate, and fixed.
- **PDF formatting nits:** a raw § symbol and insufficient spacing under a heading, corrected on the author's explicit request.

4. Validation Approach

Every claim about Restful Booker's actual behavior — auth response shape, status codes on protected endpoints, delete semantics, filter behavior — was verified against the running local instance via direct HTTP requests or actual Playwright runs before being encoded into a test assertion or written into the bug report, rather than assumed from general knowledge of "typical" REST APIs. Similarly, the CI/CD pipeline's behavior was confirmed against a real, completed GitHub Actions run (checked job-by-job via the GitHub API) rather than

treated as correct on the basis of a committed workflow file alone.